VASTRANGAM

Medhava

One Business Operating System. Any trade. One shared data core.

22 modules · 113 apps · 16 working today · compiled 2026-08-23

What this document is

Two documents in one, because they answer two different questions and people ask both.

Part One — The System is the reader's tour: what Medhava is, how one order moves through it, every module and every app, how a trade is added as a row of configuration, and the rules that hold everywhere.

Part Two — The Plan of Action is the builder's document: what Medhava is, the tenancy model, the industry pack engine, the eight build phases, the free-first tool register, onboarding, security, risks and what a customer pays for.

Both are generated from `brand/site/modules.js`, the one canonical list. Neither this page nor either part contains a module count, an app name or an app order typed by hand — which is why they cannot disagree with each other or with the software.

Where the build actually stands

16 of 113 apps are working today. Each one is a real single-file application that carries its own self-tests and passes a click-through audit in both editions. Every other app in this document is marked **designed, not yet built**, and is described as a specification rather than as something you can open. Nothing here is described as finished that is not.

The claim, and where it is checked

"Any industry" is a statement about the code, so it is checked in the code.

A trade is a row, not a fork. What a business calls things, the stages its work moves through, the extra fields its records need, the documents it issues and the reference data it starts with all arrive as one configuration file — and a pack may never contain executable code, invent a concept the engine does not have, extend a table that does not exist, declare money as anything but integer paise, switch off an immutable rule, or be applied in part.

	How many	The design
Industry packs shipped	6	a directory, no ceiling
Companies	as many as you have	a table; the shipped plan caps a subscription at 20, the software has none
Channels per company	as many as you sell on	a table, read from your data
Stock	one number per SKU	one number per SKU — never per channel
Group	sum minus inter-company trade	sum minus inter-company trade

`core/tests/packs.test.js` invents a **commercial laundry** during the test run — a trade that appears nowhere in this software — loads it from a JSON string, and requires every screen to answer in that trade's words while still refusing it the audit trail. A final assertion fails the build if the engine file ever contains a single trade word. `core/tests/core.test.js` does the matching thing for scale: ten companies with ten channels each, then eleven by eleven with no code changed.

The honesty rules this document is written under

1. Nothing is described as finished that is not. Every app is marked working today or designed.
2. No count is typed from memory. Modules, apps and build state are read from the canonical list.
3. No figure is invented. Where a rate or a price is missing, the tool posts zero and names the item rather than guessing — a guessed rate is a wrong payment to a real person.
4. Where something could not be verified, it says so instead of implying it was.

PART ONE — THE SYSTEM

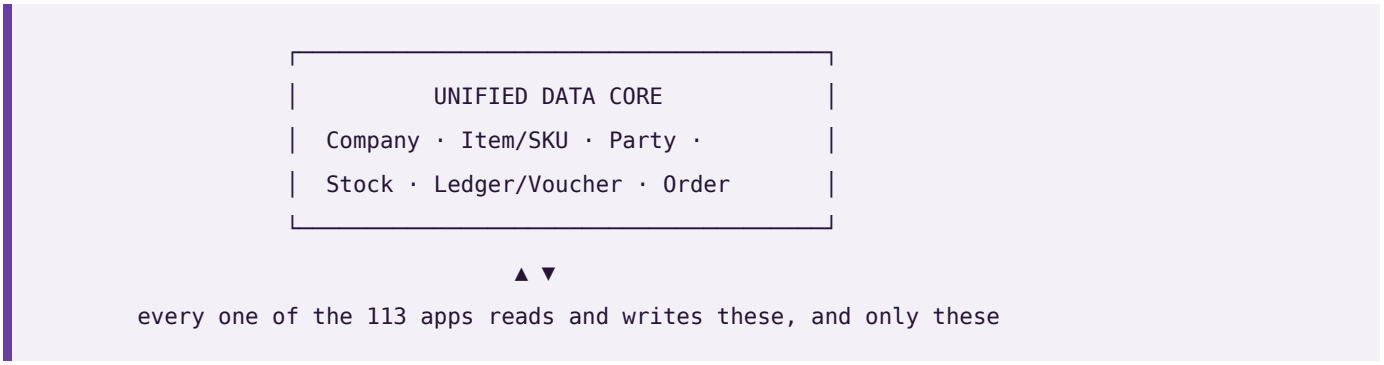
One Business Operating System for any trade: 22 modules and 113 apps over one shared data core.

This file is the whole system in plain text — every module, every app, and what each one reads and writes. It is generated from `brand/site/modules.js`, the same file the website and every PDF read, so nothing here can disagree with them. The counts below are not typed in; they are counted from that file each time this page is built.

Modules	22, built in dependency order — a module is only built once everything it needs exists
Apps	113
Working today	16 — each opens in a browser, carries its own self-tests and passes the click-through audit in both editions
Engine working, screen to come	2 — the arithmetic is written and passing its own tests on the command line; there is no screen on it yet, so it is not counted above
Still to build	95
Companies	As many as you have. A company is a row, not a setting — the shipped plan caps a subscription at 20 and the software itself has no ceiling
Shared data core	Company · Item/SKU · Party · Stock · Ledger/Voucher · Order
Key difference	Not a suite of integrated apps. One application over one database, so there is no sync step and no second copy of any master record
Compliance	Double-entry books with CGST/SGST/IGST, TDS, TCS, input credit on **accepted** goods, GSTR-1 and GSTR-3B, filed per registration
Channels	Any storefront, marketplace, counter, wholesale desk or export buyer — read from your own data, never from a list inside the code
Security	Row-level isolation per company; outside services connect with scoped, revocable keys — **never account passwords**
Deployment	Hosted, or single-file apps that run by double-clicking with no install and no internet

The one idea

Every module reads and writes the **same six records**. That is the physical reason a single goods receipt can touch stock, the books, quality and sourcing in the same instant.



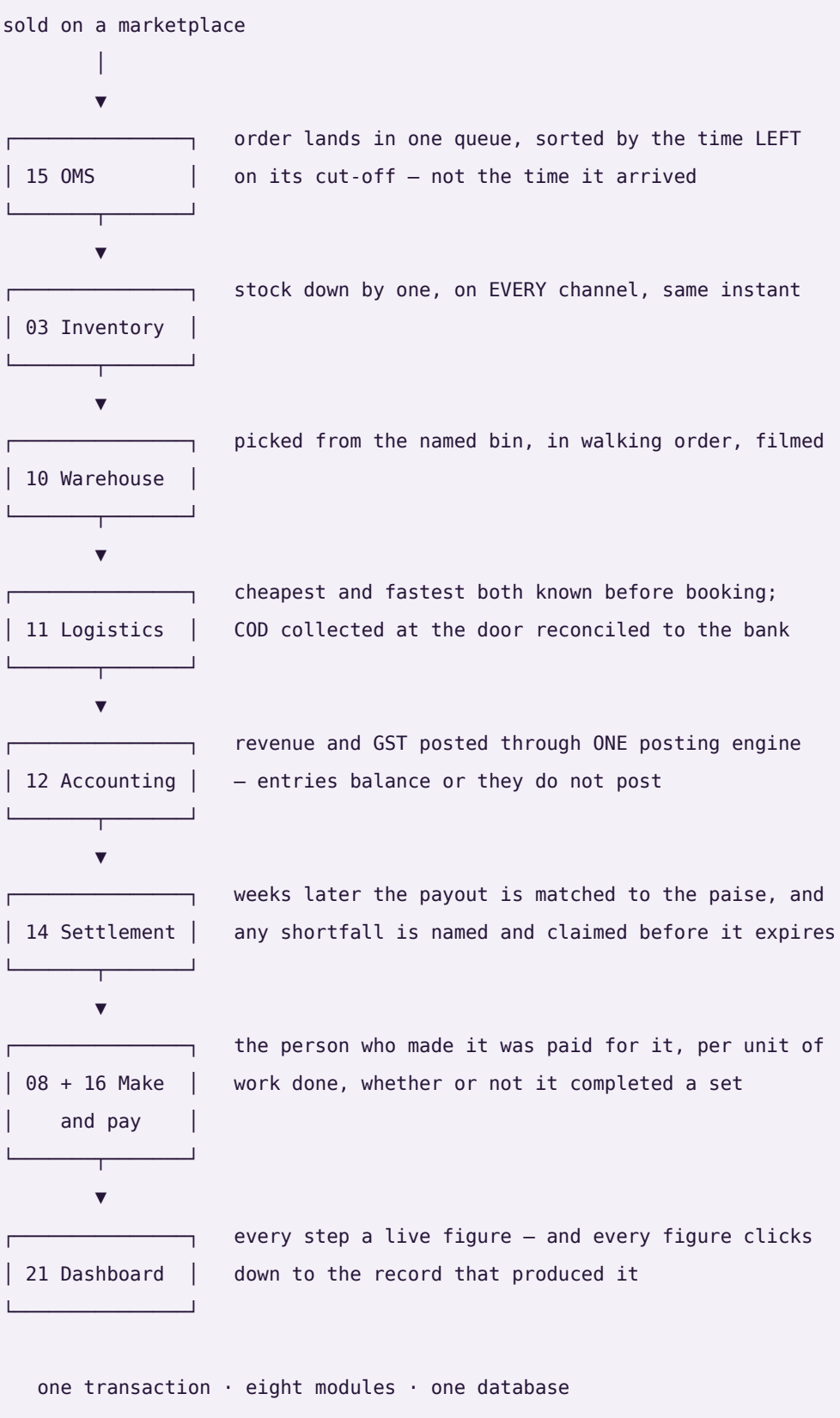
One stock number, not one per channel. The last unit sold at the counter disappears from every marketplace you sell on in the same instant — not three hours later as a cancellation, because cancellations are what a seller rating is lost to.

Accepted — not ordered — is what counts. You order 100 units. 100 arrive. Quality accepts 96. Most systems increase stock by 100 and claim tax credit on 100. This one increases stock by **96**, claims input credit on **96**, raises a debit note for the 4 rejected, and lowers that supplier's accept rate — automatically.

Nothing derived is ever stored. Outstanding, ageing, risk, promise dates, cost per piece and profit per design are recomputed on read. A stored total is a number that can drift away from the documents underneath it; a computed one cannot.

How one order moves through it

The test of whether this is one system or 113 programs sharing a login: take a single order and follow it. The words below are a product business's; a clinic, a law practice and a freight desk run the same eight modules with their own words on them.



Every industry, as a row of configuration

This is the part that makes "any trade" a fact rather than a claim. A trade is **not** a fork of the software, a branch, or a bespoke build. It is a file: what this trade calls things, the stages its work moves through, the extra fields its records need, the documents it issues, which discretionary rules apply, and the reference data it starts with.

Rank	Pack	Sector	Its words for customer · order · worker	Pipelines	Documents
1	`manufacturing`	Manufacturing	buyer · sales order · operator	2	4
2	`wholesale-distribution`	Distribution	dealer · sales order · salesman	2	5
3	`retail-ecommerce`	Retail	shopper · order · associate	2	4
4	`professional-services`	Services	client · matter · fee-earner	1	4
5	`healthcare-clinic`	Healthcare	patient · appointment · clinician	2	4
6	`logistics-3pl`	Logistics	shipper · consignment · driver	2	5

The order is not taste. Manufacturing is the largest ERP user base — around a fifth of all users and roughly a third of market revenue. Professional and financial services is next at 13.86%, and has no stock at all, which makes it the hardest case for the claim. Distribution is 9.90%. Retail and e-commerce is the largest warehouse-management segment at about 28%. Healthcare is under 5% of ERP users today and the fastest-growing of them all at 22.37% a year. Transportation is the most-served market among third-party logistics providers at 90%, and **order management ranks first** among the technology services those providers offer.

What a pack may never do. A configuration file that can do anything is not configuration, it is a hole. A pack may not contain executable code at any depth, invent a concept the engine does not have, add a field to a table that does not exist, declare money as anything but integer paise, switch off an immutable rule — company scoping, the audit trail, the posting rules, group elimination, roster privacy — or be applied in part. Each of those refusals is a named test.

One default worth stating on its own: a rule a pack never mentions is **on**. The rulebook is the default and a pack is an exception list, never a permission list. The other way round, every rule added after a pack was written would silently apply to nobody using it.

The test that decides whether this is a product. `core/tests/packs.test.js` invents a **commercial laundry** — a trade that appears nowhere in this software, in no pack, in no module and in no rule — hands the engine a JSON string while the tests are running, and requires the whole system to answer in that trade's words: an order reads as a docket, a work order as a wash load, a customer as an account. Its pipeline resolves ordered and terminating, its fields land on real tables, its rule switches resolve against the real rulebook, and it is refused the audit trail exactly as the shipped packs are. A final assertion fails the build if the engine file ever contains a single trade word — because an engine that knows one trade's words has an opinion about which trades are normal.

And the product screens are drawn from 12 sectors, not one: Drone & precision manufacturer, Freight forwarder, HVAC service firm, Law practice, Restaurant group, Interior contractor, Homeware brand · D2C, Precision components maker, Multi-doctor clinic, Training institute, Creative agency, Dairy co-operative. The same module is shown with each of their figures, one under the other, so the argument is made where it can be checked rather than believed.

Every module and every app

Listed in build order. Each app is marked **working today** or **designed, not yet built** — nothing is described as finished that is not.

Module 01 · Platform

The spine every module runs on.

Not a module you open — the layer underneath all 22. Who can see what, how the system is configured, and a record of everything that ever happened. Foundation first: nothing downstream can be built before identity, roles and the audit trail exist.

Reads from: Every module **Writes to:** Every module

App	What it does	State
Identity, Settings & Audit	Users, roles and permissions, company switching, tax and numbering setup — and an immutable record of everything that ever happened.	designed, not yet built
Industry Packs	What your trade calls things, the stages your work moves through, the extra fields your records need, the documents you issue and the reference data you start with — all of it loaded as a row of configuration, never as a separate version of the software. Pick the pack that matches your trade and the whole system changes its words: the same order screen reads as a job, a matter, a consignment or an appointment, with the same columns underneath. A pack may rename what exists, add fields to tables that exist, and switch discretionary rules off; it may never invent a concept, add a field to a table that does not exist, contain any executable code, or switch off the guarantees — company scoping, the audit trail, money never being a float — because a trade may change its vocabulary and may not opt out of the things the books are trusted for. Adding a trade nobody anticipated is therefore a file somebody writes, not a release somebody ships.	designed, not yet built
Ask & Print	Ask from your phone, anywhere: a ledger, a bill, today’s packing slips. It comes back as a PDF — or prints on the office printer, with nothing plugged into your phone.	working today
Communications	WhatsApp commands and broadcasts, email and SMS, and the handful of scheduled jobs that carry a nudge to the right person without anyone remembering to send it. Every capability here has more than one interchangeable provider — none of them is ever the source of a figure the business reports.	designed, not yet built
WhatsApp Command Console	The shop floor does not open a laptop. A worker or a staff member sends a short message — in, out, today’s pieces, leave, an advance — and it becomes a real record: attendance with the time and place it was marked, a production report against the design, a request in the approvals queue. The end-of-day prompt asks what is still open and how long it needs, so tomorrow starts from what is actually pending rather than from memory. Marking attendance checks the phone is at the unit within the radius set for it, with a grace window; a person outside it is not refused, they are flagged for the manager, because a system that locks someone out of being paid for being early at the wrong gate has failed at its job. Every override is recorded with who made it. The words a worker types are in the language they speak, and nothing here ever asks for a password, a bank detail or a document number.	designed, not yet built
Data Privacy & Consent	What a person’s data may be used for, captured as consent at the point it is given and honoured everywhere downstream — including the right to have it corrected or removed. Retention (how long a record is kept) and deletion (a person’s right to have their own data removed) are two different policies, tracked separately, because a rule that keeps records for the law and a request from a person to be forgotten do not resolve the same way.	designed, not yet built

App	What it does	State
Provider Router & Cost Guard	The rule that no capability depends on one outside service, enforced at the moment it matters instead of merely promised. Every capability has an ordered fallback list ending on an option that needs nothing connected, so a courier API that stops answering at 9pm, or an AI key that hits its quota mid-catalogue, drops to the next option instead of stopping the work. A provider that keeps failing is tripped out of the list entirely and retried once after a cooldown rather than hammered; retries inside one provider wait twice as long each time. And every paid call is counted in paise against a ceiling you set — over the ceiling the paid provider is refused, not warned about, and the work completes on a free one. Because every capability is guaranteed a built-in or by-hand option, a spent budget can stop the spending without ever stopping the business.	engine working, screen to come
Payment Data Scope	A written statement of exactly which systems ever see a card or bank credential, and which never do — because every card-capable screen in this system hands that moment to a payment provider's own secured field and never stores or even passes the number through application code. The statement is what an auditor or a partner asks for before they will connect to this system.	designed, not yet built

Module 02 · Design & Sampling

A style exists on paper before it exists as stock.

A product moves from a first idea to something the business can actually make and sell — specification, sample rounds, costed trials and sign-off — before it is ever entered as a catalog record. Building this first means Inventory & Catalog never has to invent a style it has no real specification for.

Reads from: CRM **Writes to:** Inventory & Catalog · Manufacturing

App	What it does	State
PLM & Development	First idea to something you can actually make: specification, sample rounds, costed trials and sign-off, with every version kept. A product, a machined part, a formulation or a service package all move through stages you set yourself.	designed, not yet built
Design / IP Register	What protects a design once it exists — trademark or copyright status, the date it was first shown, and a flag the moment a near-identical listing turns up elsewhere. Every version already kept by PLM & Development gets a filed status here instead of just a date stamp, because a design with no ownership record on file is a design nobody can defend.	designed, not yet built

Module 03 · Inventory & Catalog

One number everyone trusts.

The most important number in the system: one quantity per SKU, per location, per stage — read and written by every other module. And one product record that every channel lists from.

Reads from: Design & Sampling · Every module **Writes to:** Every module

App	What it does	State
Stock	Live quantity by SKU, location and stage, with reorder alerts, batches, kits and dead-stock. Goods you still own but that sit in a channel's own warehouse are a location like any other, so consignment and sale-or-return stock is counted, valued and aged with everything else instead of disappearing off the books until it sells.	designed, not yet built
Catalog / PIM	One product record — attributes, images, HSN, MRP and the price each channel actually sells at — pushed to every marketplace and to your own storefront, and scored for each channel's rules before it lists. It also holds the two things everything downstream depends on: the code each channel knows this product by, mapped to yours, and the packed size and weight that decide the courier rate and settle every weight dispute. Every listing's state on every channel is here too — live, waiting for your approval, blocked, archived — with the quality score that decides whether anyone sees it.	designed, not yet built
Kit & Combo SKU	A sellable SKU made of component SKUs — a three-piece set sold as one listing. Selling the kit decrements each component at order time, so stock is right for every piece in the set, not just the set itself.	designed, not yet built
Master-Data Hygiene	Duplicate detection and merge across customers, vendors and designs, and a dead-stock register for what has not moved in months. Protects every downstream report from the same record existing twice under two names.	designed, not yet built

Module 04 · CRM

Know every customer completely — and answer them fast.

One record per customer carrying every lead, order, return, document and conversation, whichever channel it came from. Whoever picks up the next question can already see everything that came before it.

Reads from: Every module **Writes to:** Sales · E-commerce / OMS · Marketing

App	What it does	State
CRM & Customer 360	Lead to won, then the full lifetime: orders, returns, value and what to offer next.	working today
Documents & eSign	Every agreement, receipt, certificate and scan filed against the record it belongs to — an order, a party, a case, an employee — so it is found by that record instead of by remembering a folder. Send one out for signature and the signed copy files itself back.	working today
Helpdesk & Live Chat	Questions arriving by chat, email or phone become tickets tied to the order or the account they are about, with the whole history already on the screen.	working today
Forms & Feedback (NPS)	A short form after delivery, and the score it produces attached to the design or item it is actually about — not just the buyer — so a complaint-prone item surfaces as a pattern instead of a scatter of individual gripes.	designed, not yet built

Module 05 · Sales

Every way you sell, one order book — to the doorstep.

Retail counter, wholesale, export and your own website all write to the same order and draw on the same stock number. The courier side lives here too, so a sale is not finished when it is billed — it is finished when it is delivered and the COD money is in.

Reads from: Inventory & Catalog · CRM · Warehouse · Logistics **Writes to:** Inventory & Catalog · Accounting & GST · Warehouse · Logistics

App	What it does	State
D2C Sales	Orders from your own storefront — Shopify, WooCommerce or a custom site — cart to dispatch, with loyalty and partial COD.	working today
B2B & Credit	Wholesale orders with credit limits, tier pricing and outstanding ageing.	working today
Export	Commercial invoice, packing list, LUT bond and IGST-refund tracking.	working today
POS	Counter billing that draws on the same stock as your website.	working today
Quotes & Proforma	Send a quote, convert it to a confirmed order in one click.	working today
Couriers & AWB	Book the shipment on the order itself, compare couriers, print the label and follow the AWB to the door.	designed, not yet built
Subscriptions	A schedule that raises its own invoice on its cycle and follows up on its own when a payment fails — for anything sold as a standing order rather than a one-off.	designed, not yet built
Customisation & Made-to-Measure	The order that does not exist in the catalogue: a buyer sends reference pictures and their own measurements, a price is agreed over several messages, and the piece is made for them. All of it is one record — the references, the measurement set, every quote in the negotiation and what was finally agreed, the advance taken to start work and the balance taken before dispatch. When the order is accepted it opens a production order like any other, so a bespoke piece is costed, made, checked and posted exactly as a catalogue piece is. Two legs of money on one order is the part most systems get wrong: the advance is earned when the work starts, the balance is owed until the piece ships, and the ledger shows both separately rather than one payment appearing when the whole thing is over.	designed, not yet built

Module 06 · Planning & Requirements (MRP)

Turn what is selling into what to buy and make.

Confirmed orders and demand history have to become a plan before Purchase can buy anything or Manufacturing can start anything — otherwise buying and making are both just guessing. This module sits between the two: it

reads what is actually selling and what is already committed, and turns that into requirement, not the other way around.

Reads from: Sales · E-commerce / OMS · Inventory & Catalog **Writes to:** Purchase · Manufacturing

App	What it does	State
Demand Forecast & Signal	What sold, by SKU and by period, turned into a short-term forecast — the number every requisition and every production order downstream is measured against instead of a guess.	designed, not yet built
Requirement Explosion (MRP run)	Confirmed demand exploded through the bill of materials into what raw material to buy and what to produce, and by when — so a purchase requisition or a production order always traces back to real demand, never to a feeling that stock is getting low.	designed, not yet built
Open-to-Buy / Budget Ceiling	A spending ceiling set per period regardless of what the demand signal suggests, so a real signal cannot on its own commit more money than the business has decided to risk this season. Every requisition the MRP run drafts is checked against what is left of the ceiling before it goes to Purchase.	designed, not yet built

Module 07 · Purchase

Nothing over-billed gets paid.

The buy side end to end — and the control that stops you paying for goods you rejected.

Reads from: Inventory & Catalog · Planning & Requirements (MRP) · Manufacturing **Writes to:** Inventory & Catalog · Accounting & GST · Quality & Compliance

App	What it does	State
Procurement	RFQ to purchase order to goods receipt, with a strict three-way match before any bill is paid.	working today
Vendor Management	Vendor 360, payables, ageing, a real risk score, and sourcing that follows performance.	working today
Insurance Register	What cover exists over stock in transit, stock in the warehouse and product liability, matched against what is actually moving through Purchase and Warehouse right now — so real value in transit is never quietly running with no cover tracked anywhere.	designed, not yet built

Module 08 · Manufacturing

Know what a unit really costs to make.

From material in the door to the finished unit — including what every worker earned and what each product actually cost. You define the stages, the rates and the rules; nothing here is fixed to one trade. Design and sample sign-off happen upstream now, and quality inspection and the compliance record live in their own module downstream — this module is purely the making.

Reads from: Purchase · Planning & Requirements (MRP) · Design & Sampling **Writes to:** Inventory & Catalog · HR & Payroll · Accounting & GST · Quality & Compliance

App	What it does	State
Production Orders	Your own stages from first operation to finished goods, with work-in-progress visible at each one.	designed, not yet built
Piece-rate & Contractors	Output-based pay for anyone paid by the piece rather than the hour — pooled completion, per-unit rates, rework and advances resolved into a single payout.	designed, not yet built
BOM & Consumption	What each product consumes, costed at today's material rates.	designed, not yet built
Maintenance	Machines, tools and assets: what is due for service, when it was last done, what it cost, and what stopped while it was down. Planned and breakdown work against the same asset record.	designed, not yet built

Module 09 · Quality & Compliance

Certify what was received and what was made.

Quality inspection used to live buried as one step inside Manufacturing; it stands on its own here because a rejection at goods-receipt and a rejection on the production floor are the same discipline, and because the certificates a business holds — the proof it follows a standard — belong next to the inspections that back them up, not scattered across email.

Reads from: Purchase · Manufacturing **Writes to:** Purchase · Manufacturing · Inventory & Catalog

App	What it does	State
Quality Control	Accept, reject or rework — on goods received and on goods made — with reasons that feed the supplier scorecard and the performance flags alike.	designed, not yet built
Certificate & Compliance Register	Every standard the business or a vendor holds — a safety, labour or environmental certification — with its issue date, its expiry, and the audit that backs it, so a certificate about to lapse is visible before a buyer asks for it and finds it already expired. What this register tracks is what a sustainability report downstream is actually built from.	designed, not yet built

Module 10 · Warehouse

Pick right the first time — and prove what you sent.

Bin-level instructions and barcode scanning, so the right item leaves the building and stock stays honest — and a recording of each parcel being packed, so an argument about what was in it is settled by footage instead of by memory.

Reads from: Sales · E-commerce / OMS · Inventory & Catalog **Writes to:** Inventory & Catalog · Sales · E-commerce / OMS

App	What it does	State
Picking & Bins	Pick lists that tell staff exactly which bin to walk to, in walking order.	designed, not yet built
Barcode Operations	Scan to pick, pack, dispatch and run a physical stock count from a phone — the same scan whether the order came from a marketplace, your Shopify site or the counter.	designed, not yet built
Packing Video	Every parcel recorded as it is packed and indexed by its order number, so a wrong-item claim is answered with the clip. The footage attaches itself to the claim that needs it.	designed, not yet built

Module 11 · Logistics

The courier network itself — rates, failures and the COD money.

Booking one parcel happens on the order, in Sales. This module is the network behind it: what every courier charges before you pick one, what happens to a delivery that fails, and whether the cash collected at the door actually reached your bank.

Reads from: Sales · E-commerce / OMS · Warehouse **Writes to:** Accounting & GST · Sales · E-commerce / OMS

App	What it does	State
Rates & Zones	Every courier’s rate card by zone, weight slab and service — so the cheapest and the fastest option for this parcel are both known before it is booked.	designed, not yet built
NDR & RTO Rescue	A failed delivery worked while it can still be saved — reattempt, call, correct the address — before it becomes a return you pay for twice.	designed, not yet built
COD Remittance	What the courier collected at the door against what reached your bank, parcel by parcel, with every shortfall named and aged.	designed, not yet built
Handover & Manifest	What is expected out today against what the courier actually took, counted per courier and per service. The manifest to hand over, the one-time code to confirm it, and a signed record of the parcels that were left behind — so a parcel lost between your table and their van has an owner.	designed, not yet built
Fleet	Your own delivery vehicles, if you run any — what each trip cost, what is due for service, and what that adds to freight. Optional; most businesses run couriers only.	designed, not yet built

Module 12 · Accounting & GST

Books that always balance.

A full double-entry ledger built for Indian compliance — not a tax report bolted onto a spreadsheet. Medhava keeps the books on its own: no other accounting package is required, ever.

Reads from: Every module **Writes to:** Finance Reports · Treasury & Financial Planning

App	What it does	State
Accounting	Double-entry books where every voucher balances and the trial balance always ties.	designed, not yet built
Invoicing	GST tax invoices and receipts, totals computed from the lines to the paise. Where a channel raises its own invoice, both numbers live on the order — theirs and yours — so the panel's paperwork and your books point at the same sale and neither has to be re-keyed to find the other.	designed, not yet built
Expenses	Spend captured by category with approvals, and bill OCR to save typing.	designed, not yet built
GST & Tax	CGST, SGST, IGST, TDS, TCS, input credit, GSTR-1 and GSTR-3B — filed per registration, so a group where one company is registered and another is not files exactly what each one owes and nothing it does not.	designed, not yet built
ITC Reconciliation	Every input credit matched against the government's own GSTR-2A/2B before a return is filed, so a credit claimed is a credit that is actually on record.	designed, not yet built
Receivables, Payables & PDC	Payments and receipts allocated against named open invoices, and a register for post-dated cheques that posts only on the date they are realised, not the date they were written.	designed, not yet built
Fixed Assets & Depreciation	The asset register with both Straight-Line and Written-Down-Value depreciation tracked side by side, and disposal posting its gain or loss straight to the books.	designed, not yet built
Year-End Close & Period Lock	Profit-and-loss accounts reset and balance-sheet accounts carry forward at year end, and a reviewed period locks against backdated edits until an admin unlocks it — an act that is itself logged.	designed, not yet built
Finance Reports	P&L, balance sheet, and profit by channel, product and SKU.	designed, not yet built

Module 13 · Treasury & Financial Planning

Know what cash is coming, not just what already arrived.

Accounting records what happened; this module is concerned with what happens next — how much cash is actually expected, when, and whether spend against a budget is on track before the month closes and turns the answer into history.

Reads from: Accounting & GST · Sales · Purchase **Writes to:** Accounting & GST

App	What it does	State
Cash Flow Forecast	Expected receipts and expected payments laid out by week, drawn from open invoices and open bills rather than typed in by hand — so a cash shortfall is visible weeks before the date it would actually bite.	designed, not yet built
Banking & Reconciliation	Bank statement lines matched against the ledger's own record of what should have moved, with anything unmatched surfaced instead of silently carried forward.	designed, not yet built
Budget vs Actual	A budget set per category and period, and actual spend from Accounting tracked against it as the period runs — not only after it closes, when the only thing left to do is explain the variance.	designed, not yet built

Module 14 · Settlement

Get paid what you are owed — cycle by cycle.

Matching one payout to one order line happens in OMS. This module is the level above it: the settlement cycles each panel runs, the fees it actually charged against the fees it published, and the tax it deducted on your behalf.

Reads from: E-commerce / OMS · Accounting & GST **Writes to:** Accounting & GST

App	What it does	State
Payout Cycles	Every settlement cycle each panel runs — what it should pay, what actually landed in the bank, and on which day — so a late payout is visible the day it is late, not at month end.	designed, not yet built
Fee & Commission Audit	The rate card a channel publishes against the rate it actually charged, category by category and SKU by SKU. A silent commission increase is caught the first time it is applied — and the tier you are rated in is on the same screen, because the tier is what the rate card hangs off, and losing one quietly costs more than any single deduction.	designed, not yet built
TCS & TDS Register	Every rupee the panels deducted as TCS and TDS, matched against the portal's own figures — so the credit you claim is the credit you are actually owed.	designed, not yet built

Module 15 · E-commerce / OMS

Every marketplace and your own website, one queue.

Stop logging into seven seller panels and your own store admin. Every order — Amazon, Flipkart, Meesho, Ajio, Nykaa, JioMart, Myntra, and your Shopify, WooCommerce, Magento or custom site — lands in one pipeline, and one stock number goes back out to all of them. Then the money side closes in the same module: what each channel paid, what it kept, what came back, and what you are still owed.

Reads from: Inventory & Catalog · CRM · Sales · Accounting & GST · Logistics · Settlement **Writes to:** Inventory & Catalog · Accounting & GST · Warehouse · Logistics · Settlement

App	What it does	State
Marketplace OMS	Every marketplace and every storefront in one order queue — Amazon, Flipkart, Myntra, Meesho, Ajio, Nykaa and JioMart alongside Shopify, WooCommerce, Magento, Wix and your own custom site. The stages each channel really uses — to accept, to pack, ready to dispatch, handed over, in transit — with the right cut-off counting down on every order, because a quick-commerce or air-shipped order is not due at the same hour as a standard one. Priority orders first, the day grouped by product so one item is picked once instead of once per parcel.	working today
Order Management	One pipeline from new to delivered, whether the order came from a seller panel, your Shopify or WooCommerce site, a dealer or the counter.	working today
Manual Data Check	Upload the sheets you already download — marketplace orders and returns, and your own counter-shop registers, one file or a whole ZIP — and read ten cross-checks back: money, month, item, state, returns, claims, ads, payouts and GST. Every figure is clickable down to the transactions behind it, and the whole result downloads as Excel.	designed, not yet built
Reconciliation	Match every marketplace payout to the order line that earned it, and expose the gap.	designed, not yet built
Claims & Disputes	Turn shortfalls, weight disputes and lost parcels into filed claims with evidence — and answer them before the clock runs out. A claim that is awaiting your response is worth money; one closed for no response is worth nothing, so the days remaining sit on the screen next to the amount.	designed, not yet built
Returns / RMA	Customer, courier and wrong returns — and the dead stock they actually cost you.	designed, not yet built
Channels & Storefronts	Connect a channel once and it stays in step: catalogue out, price out, stock out, orders in. Seven marketplaces and any website platform — Shopify, WooCommerce, Magento, BigCommerce, Wix or a custom site over its own API — each switchable without touching your data. Shopsy and any other storefront a channel runs alongside its main one counts as its own channel here. Where a channel has no open interface, its own downloaded report is a first-class way in. A channel may also know you by a different trading name — that is a label on the channel, not a second company, so it tags the order and the payout without ever splitting your books.	designed, not yet built
Labels & Documents	The channel gives you a PDF; this turns it into something a packer can work from. Cropped to your label size, your own product code printed large where the channel left it off, the invoice and the packing slip merged behind it, and the whole batch sent to the label printer in one job. Reprint a single parcel without redoing the batch — and nothing is ever uploaded to an outside website to be cropped.	designed, not yet built
Listing & Catalog Manager	Bulk-create and bulk-edit listings across every channel from the one product record in Inventory & Catalog, and catch the mismatches that quietly cost sales: listed but out of stock, or in stock but never listed.	designed, not yet built
Size / Fit Recommendation AI	A fit suggestion at the point of purchase, built from the item’s own measurements and the return history of buyers who picked each size — aimed straight at the return reason that costs the most: the right item in the wrong size.	designed, not yet built

App	What it does	State
AR / Virtual Try-On	A way to see drape, fit and colour on a screen before buying, for items where a flat photo alone leaves too much to guess.	designed, not yet built

Module 16 · HR & Payroll

Pay people right, on time.

Salaries and output-based earnings in one register, with attendance driving both — whether people are on a monthly wage, an hourly rate or paid by what they finish.

Reads from: Manufacturing **Writes to:** Accounting & GST

App	What it does	State
Staff & Contractors	Attendance, effective-dated salary and output-based earnings in a single register, whoever is on it.	designed, not yet built
Time-off & Advances	Leave, festival advances, and exactly how they change this month’s payout.	designed, not yet built
Appraisal & Hiring	Performance reviews and a hiring pipeline that ends in an employee record.	designed, not yet built
Recruitment	The pipeline before someone becomes an employee — an opening, the people who applied for it, a trial piece where the work itself is the interview, and the decision with its reason kept. It matters more in a skilled trade than in most: a person is taken on for skill at one particular kind of work, and the trial output is the evidence, so it is recorded against that work and the rate that would apply rather than remembered as an impression. A candidate who is not taken on now stays findable when the same skill is needed in a busy month, and their personal documents are held under the same consent and retention rules as anyone else’s, not in a folder on somebody’s phone.	designed, not yet built
Payout Execution	Where the calculation in the earnings register actually turns into money leaving the business — bank batch, UPI, cash against a signed receipt — with the method and the reference recorded against every payout, so the register’s total and the money that actually moved can always be checked against each other.	designed, not yet built

Module 17 · Marketing

Sell more without discounting.

Plan content, run campaigns, and let rules keep your prices competitive while protecting margin.

Reads from: Inventory & Catalog · CRM **Writes to:** Sales · E-commerce / OMS

App	What it does	State
Social Calendar	Plan and publish across every channel from one calendar.	designed, not yet built
Campaigns	Email, SMS and WhatsApp campaigns measured on real revenue, not opens.	designed, not yet built
Repricing Engine	Rules per channel and SKU — floor, ceiling, match-lowest, festival overrides — and what each change actually did. A price that went up and took the orders down with it shows as exactly that, next to the rule that raised it, so the rule can be reversed on evidence rather than on a feeling.	designed, not yet built
Automation	If this happens, do that — across any module, without writing code.	designed, not yet built
Blog & Pages	Articles, landing pages and category copy written, scheduled and published straight to your own site — Shopify, WooCommerce, Magento or a custom CMS — with the meta title, description and internal links set before it goes out.	designed, not yet built
Events	Trade shows and exhibitions worked as a channel of their own — booth, budget and every lead captured on the floor landing straight in CRM instead of on a stack of business cards.	designed, not yet built
Website & Page Builder	The storefront itself, built by dragging sections into place rather than by editing a theme file — hero, product grid, size guide, lookbook, contact form — each block reading live from the catalogue, so a price or a stock state on a landing page is the same number the order screen uses instead of a figure someone pasted in and forgot. Blog & Pages above writes articles into a site that already exists; this is for the businesses that do not have one, and it is the gap that shows up plainly when this module list is set beside a mature open-source ERP: they ship a full site builder next to the blog, and until now this did not.	designed, not yet built
Markdown / Clearance Optimization	The same rule engine that reprices for competitiveness, aimed at ageing stock instead: when to start discounting it and by how much, before it becomes a warehouse write-off rather than a sale at a lower margin.	designed, not yet built

Module 18 · AI Content Engine

Write it, shoot it, cut it — from the catalogue you already have.

Listings, ads, email, product photography and reels, all generated from your own catalogue — so the words match the product and the picture is the right size for the channel it is going to. Words, images and video sit in one module because they are one job.

Reads from: Inventory & Catalog **Writes to:** Marketing · E-commerce / OMS

App	What it does	State
Content Engine	Fourteen stages in your own voice, from buyer research and competitor reading through hooks, channel-ready listings, ads, social posts, video scripts, song lyrics, the publishing calendar and alt text — each written from your own catalogue, so the words match the thing.	designed, not yet built
Image Studio	Layers, free transform, background removal, channel presets and SEO alt text — a phone photo becomes a channel-compliant product image.	designed, not yet built
Video Studio	Text and image to video, reels and ad cuts sized for every channel.	designed, not yet built
Design Studio	A full design surface — templates, layers, undo and redo, any colour, exact sizing, background images and stock elements — exporting PNG, JPG or PDF at whatever size the channel or the printer asks for.	designed, not yet built
Motion Renderer	A reel rendered from a page of HTML and CSS — the same layers, fonts and brand colours the Design Studio already uses — into a real MP4, on this machine, with nothing uploaded. It is not a screen recording: the clock is faked, the animation is seeked to the exact instant of each frame, and only then is that frame captured, so the render does not care whether the machine was busy. Rendering the same festival banner twice produces the same file to the byte, which is what makes a reel something you can check and re-cut rather than something you have to watch all the way through and hope about.	engine working, screen to come
Narration Studio	A voice over the reel, in the language the buyer actually speaks — the same script the Content Engine wrote, spoken. The default needs nothing installed and no key, because every modern browser can already speak; a self-hosted or cloud voice sits behind it as an interchangeable provider for anyone who wants a cloned or branded one. Long scripts are split at sentence boundaries and rejoined, so a two-minute description is not cut off at whatever limit a service imposes. A voice cloned from a real person is only ever used with that person's recorded consent, filed against them in Data Privacy & Consent like any other permission.	designed, not yet built
Image Generation Slot	Generated imagery — a model on a background you do not have to shoot, a festival backdrop, a lifestyle scene — as a capability with interchangeable providers rather than a bet on one service: a queue of jobs, a preview while it works, inpainting to fix one region, upscaling and face correction. This one needs a graphics card. Image models cannot run on an ordinary office machine or on the container this system is built in, so what ships is the queue, the review screen and the provider slot, with the generating itself done by whichever engine you point it at — your own GPU box, or a cloud service, swapped without touching anything else. Said plainly here because the alternative is a screen that looks finished and produces nothing.	designed, not yet built
Publisher	One push sends the finished listing, picture and copy everywhere it has to appear — your storefront, each marketplace, each social account — and reports back what actually went live and what was rejected, with the reason.	designed, not yet built

Module 19 · SEO, AEO & AIO

Be found by a search box, an answer box and an AI.

Content already exists once this module is reached; here it is made findable — by a traditional search engine, by the answer box above the results, and by the AI assistants now answering shopping questions directly instead of

sending someone to a results page.

Reads from: Inventory & Catalog · AI Content Engine **Writes to:** Marketing

App	What it does	State
Technical SEO & Schema	Structured data, sitemaps and page-level technical checks against your own storefront and content pages, so a search engine can read what a page is actually about instead of guessing from the text alone.	designed, not yet built
Answer-Engine Optimization	Content shaped to be quoted directly by an answer box or a voice assistant — a clear, citable answer near the top of the page — rather than written only for a person to scroll through.	designed, not yet built
AI-Engine Visibility Tracking	Whether and how this business is actually cited when someone asks an AI assistant a shopping question in this category, tracked over time — the same discipline as rank tracking, aimed at a newer kind of result page.	designed, not yet built

Module 20 · Projects & Collaboration

The work that is not an order — and the talking around it.

Not every business runs on orders. A law firm runs on cases, an agency on engagements, a workshop on jobs, a builder on sites. This module holds that work on the same records as everything else, so the time, the cost, the documents and the decisions attached to it end up in the books rather than in somebody’s inbox.

Reads from: CRM · Sales · HR & Payroll · Inventory & Catalog **Writes to:** Accounting & GST · HR & Payroll · CRM

App	What it does	State
Projects & Cases	A project, a case file, an engagement or a job — whatever your work is called. Stages you define, owners, deadlines, documents, billable time and real cost, all on one record the ledger can see.	designed, not yet built
Timesheets & Planning	Who is on what this week, and the hours that actually went in — against a project, a case, a job or a machine. Billable and non-billable separated, so a rate card turns straight into an invoice and a real cost.	designed, not yet built
Approvals	One queue for everything waiting on a yes — a purchase order, a discount, a leave day, a credit note, a payment. The rule that sent it there is on the screen next to it, and the decision goes to the audit record.	designed, not yet built
Forum	Questions and answers that outlive a chat — for customers, dealers or staff — with the useful ones kept where the next person will actually find them.	designed, not yet built
Automation Studio	The place a person builds “when this happens, do that” by dragging it out and watching it run — a trigger, the steps after it, a branch where the answer decides which way to go — over the same event stream every module already writes to. Marketing’s Automation aims that idea at campaigns; this is the general one, reaching any module: a payment marked short holds the next dispatch and opens a claim, a worker’s pooled output crossing a threshold raises the payout for approval, a return marked damaged writes off the piece and messages the buyer. Every run is kept — what fired it, each step, what each step returned — because an automation nobody can inspect afterwards is a rule the business cannot trust with its money.	designed, not yet built
Discuss	Conversation attached to the record it is about: this order, this bill, this case. A year later the reason for a decision is still sitting next to the decision.	designed, not yet built
Knowledge Base	A searchable internal wiki of standard operating procedures, scoped to the role it applies to, so how a task is meant to be done is written down once instead of carried in one person’s head.	designed, not yet built

Module 21 · Dashboard & BI

See the whole business without asking anyone.

Every number in Medhava rolls up here as work happens — no exports, no waiting for month-end, no asking three people for their sheet. It is the last module built for a reason: it has nothing to show until the other twenty are producing real records for it to read.

Reads from: Every module **Writes to:** —

App	What it does	State
CEO Dashboard	Cash, sales, stock, profit and alerts on one screen, refreshed as work happens.	working today
Report Builder	Drag the fields you want into a report and save it for the whole team.	working today
Group Consolidation	Several companies, one set of figures — sales, cash, stock and profit rolled up across every company you run, inter-company entries removed, with years of history to compare against. Add a company whenever the business grows one; nothing in the software caps the number, only the plan does. And a company with no tax registration of its own — a job-work arm, a new venture not yet registered — is a company like any other here, kept in the group figures without being dragged into a return it does not belong in.	working today
Excel Dashboard Builder	A full workbook — financial summary, HR, purchase, sales, inventory and production, GST, expenses — generated from the live records behind every other screen, with each company shown as its own row and a consolidated row that is a formula over them, never a separately typed total.	designed, not yet built
ESG / Sustainability Reporting	Water usage, chemical compliance, waste and packaging, reported from the same certificate and audit records Quality & Compliance already keeps — so a sustainability report is a query over evidence already on file, not a separate exercise assembled once a year from scratch.	designed, not yet built

Module 22 · AI Assistant, Agents & Automation

Ask the business a question — and let the routine work run itself.

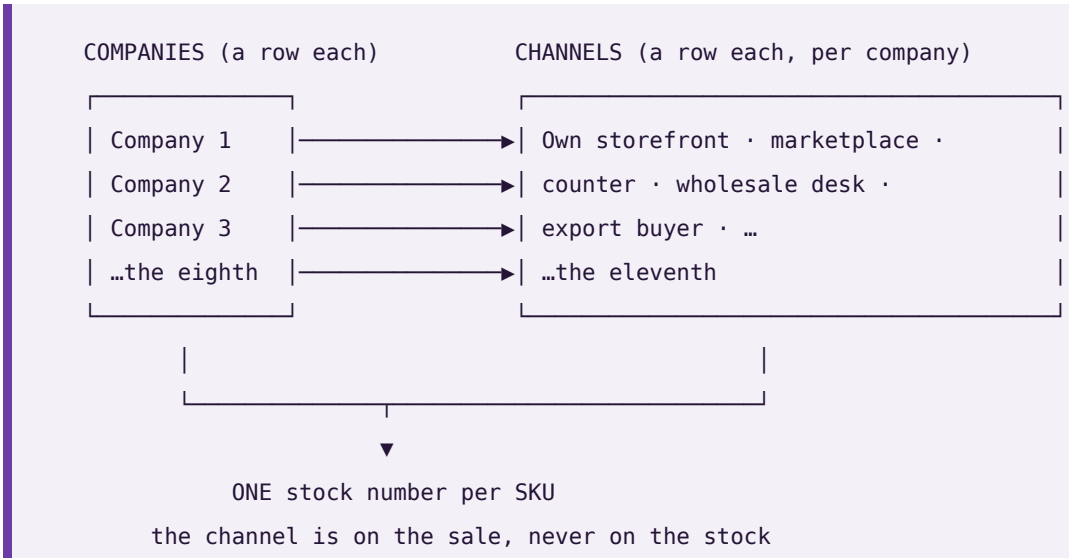
Last for the same reason Dashboard & BI is late: something that answers questions about the whole business can only be built once the whole business is in one place. Three different things live here and the difference between them matters. An ASSISTANT answers a question you asked, from the records, with the records attached. A CHATBOT holds the same conversation with your customer instead of you. An AGENT is given a job rather than a question and works out the steps itself. That last one is what separates this module from Automation Studio in Module 20, where a person draws the steps in advance and the rule runs the same way every time; and from Automation in Module 17, which fires marketing campaigns and nothing else. Both of those stay exactly as they are — this module sits above them and calls them, rather than replacing either. Module 18 writes content; this module answers and acts.

Reads from: Every module **Writes to:** Projects & Collaboration · CRM · Marketing

App	What it does	State
AI Assistant	Ask in your own words — “what did Myntra actually pay us last week, and what is still short?” — and get the answer with the rows it came from sitting underneath it, each one clicking through to the record. It reads the ledger, the stock table and the settlement lines the same way a report does, so the figure it gives is the figure the books give. When it cannot find the answer it says so and shows what it looked at; it never estimates a number and presents it as a fact, because a plausible wrong figure is far more expensive than an honest blank. It answers only from records the person asking is already allowed to open, so it can never become a way around permissions.	designed, not yet built
AI Chatbot	The same engine turned to face the customer, on your own storefront and on WhatsApp: where is my order, will this size fit me, I want to return this. It reads the real order and the real size chart rather than a script written six months ago, and it will say “let me get someone” instead of guessing at anything about money, a refund or a complaint. The handover goes into the Module 04 Helpdesk queue with the whole conversation already attached, so the person picking it up starts where the customer left off instead of asking them to explain again. It never asks a customer for a card number, a bank detail or a password — that promise does not get a chatbot-shaped exception.	designed, not yet built
AI Agents	A job rather than a question: “chase every unreconciled settlement line from last week and draft the claim for each.” The agent works out the steps, does them, and stops at the point where a person has to decide. It runs inside a scope you set — which records it may read, which it may write, and how much it may spend through the Module 01 Provider Router — and it cannot quietly widen that scope mid-run. Anything that moves money, files a claim, changes a price or sends a customer a message waits for a human yes.	designed, not yet built
Agent Guardrails & Run Log	What each agent is allowed to touch, written down as a scope rather than trusted to a prompt, and every run recorded step by step: what started it, what it read, what it proposed, what a person approved, what it actually changed. Kept in the same audit trail as everything else, with the same absence of an off switch. An agent whose working nobody can inspect afterwards is not a colleague, it is an unexplained entry in the books.	designed, not yet built
Knowledge & Retrieval	The index that makes the answers grounded: your own designs, rate cards, settlement files, standard procedures and past decisions, searchable so a reply quotes what is actually on file instead of what a model remembers about the trade in general. Permission-scoped at the row, so two people asking the same question get answers drawn only from what each of them may already see.	designed, not yet built

Companies and channels — how many is up to you

Whatever number you have, it is not a setting this system was built around, and it is not a ceiling. A company is a **row**. A channel is a **row**. Every business record carries the company it belongs to, and every sale carries the channel it came through. Ten companies selling on ten channels each is the same three tables and the same code as one company selling on one.



Three things this buys you, and one it deliberately refuses.

Each company's books are its own. Its trial balance balances on its own. No report can reach across into another company's rows — not by convention, but because a journal line can only point at an account that belongs to the same company, and a test checks that no line anywhere ever does.

The group is the sum minus what you sold yourselves. When one company in a group sells to another, that is revenue in one set of books and cost in another. Adding the companies up would report a group turnover the group never earned from the outside world. Every entry that names a sister company is eliminated at group level, and the consolidation returns all three numbers — gross, eliminated, group — so you can see the elimination rather than take it on trust.

The channel is a dimension of the sale, never of the stock. You can read this month by channel, by company, or by both. What you cannot do is keep a separate stock number per channel, and that is on purpose: the last unit sold on one marketplace has to vanish from the others at that instant, which per-channel inventory cannot do.

And the reporting follows your own sheets. The report's columns come from the sheets that are actually in the workbook you give it. A fourth company is a new sheet, not a new version of the software.

> **This is checked, not claimed.** `core/tests/core.test.js` *builds ten companies with ten channels each — a hundred channels — posts an order down every one plus ten inter-company sales, and asserts every company's books balance, that no journal line points at another company's account, and that the group figure is the plain sum* **minus** *inter-company trade: ₹2,10,500 gross, ₹50,000 eliminated, ₹1,60,500 group. It then calls the same builder for eleven companies and eleven channels with no code changed.*

The rules that hold everywhere

1. **No app depends on any single outside company.** Every capability — books, marketplaces, AI writing, couriers, payments, messaging, storage, GST, printing, barcode — has several interchangeable providers and a by-hand option, so the system works with nothing connected at all. **A provider named as the source of a figure is a bug**; providers move messages and money, the ledger originates numbers.

2. **The books are this system's own.** No other accounting package is required, ever. Tally, BUSY and Zoho remain available for anyone already running one — nothing assumes them, and no figure is ever sourced from one.
3. **Nothing ever asks for an account password.** Outside services connect with a scoped, revocable key. *This system will never ask you for a marketplace, bank or account password. If any screen ever does, it is not this system.*
4. **Money is integer paise, never a floating-point number**, because float arithmetic accumulates the error that eventually shows up as a trial balance that will not tie.
5. **Values that change over time are effective-dated** — a salary, a price, a tax rate, a commission. March payroll resolves the salary in force *in March*. A missing value is an error, never silently treated as zero.
6. **Nothing is deleted, only deactivated.** A person who leaves keeps their name attached to years of earnings, approvals and audit rows that must still resolve.
7. **The audit trail has no off switch.** Eight years, before-and-after values, as the MCA rule requires — because an audit trail that can be switched off is one that gets silenced exactly when it matters.
8. **Gates, not warnings.** Each app refuses one thing outright, because a warning gets clicked through on a busy afternoon. A cash-on-delivery order cannot be packed below its advance. A bill for more than was accepted cannot be paid. Every gate is also a self-test.
9. **It is not trained. It is built.** No model learns from your data. Every rule is written down, visible on the Wiring screen, and checked by a self-test — so it is right on day one.
10. **You can reach it from anywhere, but it cannot be reached into.** Ask & Print takes a plain line from your phone and sends a PDF back. The office reaches out; the internet never reaches in.

How it is verified

Nothing ships because it looked right on a screen.

1. **The arithmetic, with no screen involved.** Each engine runs in isolation and its self-tests execute against seeded data.
2. **Every screen and every control, in a real browser.** Each build opens in headless Chromium; every screen is visited and every interactive control on it is clicked. Any console error fails the build.
3. **The real job, with the result asserted.** Not "does the button click" but "did the thing happen". A control that looks alive but changes nothing fails the build.
4. **Against a business's own figures.** Where a business already knows the answer, the software has to reproduce it — and where it cannot, the reason is named rather than the number quietly adjusted. In the worked implementation carried furthest, every record in the owner's hand-made reference report is placed into a bucket with a named cause — matched exactly, changed at source, rate added since, a rule that applies, or present only in a source file that has since been restructured and can no longer be read — and the check fails on any record whose difference has no explanation. A mismatch is a bug, not a rounding difference; an unreadable input is a stated limitation, not a passing test.
5. **A structural audit.** Every "comes from" on every Wiring screen must name a module that actually exists, no vendor name may ever be the source of a figure, and the app count in every file must match this one.

6. **The shipped copy, not the working copy.** The packaged archive is extracted and re-tested in the folder a customer would open it in, because a packaging step that quietly renames a file breaks nothing until it is in somebody's Downloads folder.

The honesty charter

This is the standard the build is held to, and it is written down because a standard nobody wrote down is a standard nobody can be held to.

1. **Nothing is called finished that is not.** Every deliverable is labelled tool, stub, mockup or spec. A mockup stays labelled a mockup even when a working version would be more impressive to show. Generation features stay badged as mockups until a real paid API is actually wired.
2. **Counts are counted, never claimed.** Every module and app figure on this page is read from the canonical module list when the page is built. No number here was typed by hand.
3. **Progress is reported as it is.** If tests fail, the failure is shown with its output. If a step was skipped, it is named as skipped. "Done" means implemented, tested and checked against the original request — not "the code has been written".
4. **The gap is stated, not buried.** 16 of 113 apps work today. A further 2 have a working, tested engine but no screen on it yet, and are counted separately rather than folded in to make the first number look larger. The remaining 95 are designed and specified. Those 16 still run on their own storage, and rewiring them onto the shared core is the first job of Module 01 — until that is done they are good tools, not yet one system.
5. **Uncertainty is surfaced, not smoothed over.** Where something cannot be verified, it is reported as unverified rather than presented as fact.

Medhava · one business, one brain · 22 modules · 113 apps · one shared data core · 16 working today

---

PART TWO — THE PLAN OF ACTION

One business operating system. Any industry. One shared data core.

22 modules · 113 apps · 285 rules (86 enforced) · 113 tables · 46 product screens across 12 sectors · 19 tool capabilities · 6 industry packs. Every count in this line is read from `brand/site/modules.js`, `brand/site/rules.js`, `brand/site/shots.js`, `brand/site/tools.js`, `core/schema.postgres.sql` and `core/packs/` by `brand/site/mkcounts.js` — no figure here was typed from memory.

What this document is. The plan for building Medhava as a product that any business can adopt, from planning through execution. It is not the Vastrangam plan with the names changed. `PLAN_OF_ACTION.md` is that: one ethnic-wear group in Surat adopting this engine, with its own karigar payroll, its own three companies, its own migration off an accounting package. That document stays as it is, and it is the most useful thing in this repository — it is the

worked example of the whole exercise, an entire industry implementation carried to the level of detail that proves the engine bends.

This document answers the different question: **how does the same engine serve a dental clinic and a freight forwarder and a dairy co-operative, without becoming a different program each time?**

PART I — WHAT MEDHAVA IS

M1 · ONE ENGINE, MANY TRADES

Medhava is a single application over one shared data core. A business event is entered once and flows everywhere it belongs — that law, and the cascades that follow from it, are set out in full in `PLAN_OF_ACTION.md` §A0 and are identical here, because it is the same engine. This document does not restate them.

What is specific to Medhava is the claim that the engine does not know which trade it is in. That claim is either true in the code or it is marketing, so here is where it is enforced:

The claim	What makes it true
No module assumes an industry	`brand/site/modules.js` is audited at build time; trade vocabulary in it fails the build
Trade words live in one place	The edition overlay may change wording only — `build.js` compares the structural shape before and after applying it and exits if a module number, an app name or an app count moved
The same structure ships twice already	Two editions build from one module list: MEDHAVA neutral, VASTRANGAM in one trade's own words
The number of companies is data	`core/tests/core.test.js` posts across a 10 × 10 company × channel grid, then runs 11 × 11 with no code changed

The sentence the whole product rests on. A clinic's appointment, a factory's work order, a law firm's matter, a contractor's site and a service firm's job are the same record with different words on it. Everything below is the work of making that true rather than clever.

M2 · THE INDUSTRY PACK ENGINE

An earlier version of this document said, in this place, that the industry pack was **specified, not built** — that a third trade meant someone writing a third overlay by hand, and that this did not scale to a product. That was the honest state of it then. It is built now, and the paragraphs below say what was built, what it refuses, and which trades it was pointed at first and why.

M2.1 · Which trades actually run this software

The pack order was not chosen by taste. It was chosen by looking up who actually buys ERP, order management and warehouse management, and building for the largest first. Published estimates differ between research

houses — sometimes considerably, because "share of users" and "share of revenue" are different questions — so the figures below are attributed, and the ranking rather than any single number is what the build order rests on.

ERP — who the users are

Industry	Share of ERP users	Note
Manufacturing	~21% of users; ~32% of market revenue	The largest block on every measure. Cited elsewhere as 19.7% of revenue — the spread is why the ranking, not the decimal, is what is used
Banking, financial services, insurance	~16%	Heavily served by specialist systems rather than general ERP
Professional and financial services	13.86%	The second-largest block, and the one with no stock at all
Distribution and wholesale	9.90%	Credit and movement, not production
Healthcare	4.95%	Small today — and the fastest-growing, at **22.37% CAGR to 2030**
Construction	1.98%	
Education	0.99%	

Finance and insurance, manufacturing and public administration together account for roughly 60% of total ERP spend.

WMS — who the users are

End user	Share	Note
Retail and e-commerce	~28%, about $1.37bn in 2025, 10.1% CAGR	The largest WMS segment
Manufacturing	23.6–30.22% depending on the source	
Healthcare and pharmaceuticals	~10%	**The fastest-growing, at 12.2% CAGR**

The whole WMS market is put at $3.4bn–$5.92bn in 2025, again depending on who is counting and what they count as WMS.

OMS and third-party logistics — which markets are served

Market served by 3PLs	Share
Transportation	90%
Manufacturing	83%
Retail	83%
E-commerce	68%
Wholesale	67% (down from 83% the previous year)

And among the technology services those providers offer, **order management ranks first** — ahead of visibility (86%), transport management (84%), optimisation (77%) and ERP integration (75%). That is the finding that matters most for Medhava's shape: for a large part of this market the order, not the ledger, is the system of record.

Sources, read 2026-08-23: Grand View Research, Mordor Intelligence, HG Insights, MarketsandMarkets, Manufacturing Lead Generation, Inbound Logistics (3PL Perspectives), Electro IQ, openpr, NetSuite. These are market-research estimates, not measurements; they are used here to order a build queue, which is what they are good for, and not quoted anywhere in the product as fact.

M2.2 · What a pack is

An industry pack is a row of configuration, loaded as data, that carries:

- **vocabulary** — what this trade calls an order, a customer, a unit of work, a stage
- **stages** — the pipeline a job moves through, named and ordered by the trade
- **fields** — the extra attributes this trade needs on a record, and their types
- **documents** — the papers it issues, and what has to be on them
- **rule switches** — which discretionary rules apply, and at what thresholds
- **starting data** — a chart of accounts, roles, units of measure

The engine reads a pack the way it already reads companies and channels: as rows. Nothing in `modules.js`, `core/` or the schema changes when a fourteenth trade is added — which is exactly the property the 10 × 10 test already proves for companies, applied to trades.

M2.3 · What a pack may never do

A configuration file that can do anything is not configuration, it is a hole. Six refusals are enforced in `core/packs.js` and each one is a named test in `core/tests/packs.test.js`:

A pack may not	Because
contain executable code, at any depth	the moment a pack can run code, adding a trade is a code change again and the whole guarantee is worthless
invent a concept the engine does not have	"vocabulary" would quietly become a place to put anything, and the screens would carry a name for something that does not exist
add a field to a table that does not exist	the shape of the database would move outside the reach of the schema test that guards it
declare money as a plain number	the entire schema was built to keep floating-point rupees out; a pack is not a side door
switch off an immutable rule	a trade may call an invoice a fee note; it may not decide its audit trail is optional
be applied in part	a half-loaded trade is a system whose vocabulary and rules disagree, with no way to tell which half is live

The rule ids marked immutable in `core/packs.js` cover company scoping, the audit trail, the posting rules, group elimination and roster privacy — and a test asserts that every one of them really exists in the rulebook, so the protection cannot silently guard nothing.

One default is worth stating on its own, because getting it backwards would be invisible: **a rule a pack never mentions is ON.** The rulebook is the default and a pack is an exception list, never a permission list. The other way round, every rule added after a pack was written would silently apply to nobody using it.

M2.4 · Which packs ship

Built in the order the research ranks them:

Rank	Pack	Sector	What it is really for
1	`manufacturing`	Manufacturing	The largest ERP user base, and the vocabulary furthest from a service firm's
2	`wholesale-distribution`	Distribution	Credit, not production, is the thing the software has to hold
3	`retail-ecommerce`	Retail	The largest WMS segment; returns and settlement are the hard part
4	`professional-services`	Services	No stock at all — the hardest case for the neutrality claim
5	`healthcare-clinic`	Healthcare	Small now, fastest-growing on every measure
6	`logistics-3pl`	Logistics	The most-served 3PL market, where the order *is* the product

M2.5 · The gate, and what it proves

Phase 2's gate was written before the engine was: *a third trade is added without writing code.* `core/tests/packs.test.js` §4 runs it, and runs it harder than the phase asked. During the test run it invents a **commercial laundry** — a trade that appears nowhere in this repository, in no pack, in no module, in no rule —

hands the engine a JSON string, and then requires the whole system to answer in that trade's words: an order reads as a docket, a work order as a wash load, a customer as an account. Its pipeline resolves ordered and terminating, its extra fields land on real tables, its rule switches resolve against the real rulebook, and it is refused the audit trail exactly as the six shipped packs are.

A last assertion closes the loop: ** `core/packs.js` may not contain a single trade word.** Not laundry, not clinic, not freight, not dealer, not matter, not patient. If the engine ever learns a trade, the test fails — because an engine that knows one trade's words has an opinion about which trades are normal, and that is the failure this whole section exists to prevent.

`node core/tests/packs.test.js` → **every check passes, 0 failures.**

M3 · TENANCY — WHAT IS A ROW AND WHAT IS CODE

Concept	Row or code	Consequence
Tenant (a customer of Medhava)	row	Onboarding a business is data entry, not a deployment
Company inside a tenant	row	A group with four companies is four rows, consolidated with inter-company trade eliminated
Channel	row	A new marketplace is a row; rule R15.1 makes discovery automatic
Location, stage, role	row	A warehouse, a production stage and a job title are all configuration
Industry pack	row	A trade is configuration, not a fork — `core/packs.js`, six packs shipped, a seventh added during the test run
The 22 modules	code	The structure is the product; it is the same for everyone
The rulebook	code	Which apply is configurable; what they refuse is not — and 22 of them cannot be switched off by any pack

Isolation. Every business table carries `company_id` and a row-level security policy carrying both `USING` and `WITH CHECK`, so a read and a write are separately prevented from crossing a boundary. `core/tests/schema.test.js` fails the build if any company-scoped table lacks one. Cross-tenant isolation is the same mechanism one level up and is the single highest-risk item in this plan — a bug there is not a defect, it is an incident.

M4 · THE 22 MODULES, READ FROM TWELVE TRADES

The module list is in `PLAN_OF_ACTION.md` Part II in full, with every app and every rule. It is not repeated here. What is here is the reading that makes it industry-neutral — the same module, seen from a different trade:

Module	Manufacturing	Professional services	Healthcare	Hospitality
02 Design & Sampling	part revision	matter scoping	protocol	recipe development
05 Sales	order book	engagement letter	appointment book	cover forecast
06 Planning	material requirement	resourcing	rota and stock	purchase plan
08 Manufacturing	work order, stages	—	—	batch and prep
09 Quality	inspection	file review	licence and calibration	food safety log
10 Warehouse	pick and pack	—	consumables	store issue
16 HR	shift and piece-rate	timesheets, chargeable	sessions and locums	rota and casuals
20 Projects	improvement work	matters	care pathways	openings

The landing page carries every one of the product screens counted at the head of this document, across all twelve sectors, doing exactly this — the same module rendered with a machine shop's figures and a clinic's figures, one under the other. That is the argument made where it can be checked rather than believed.

PART II — EXECUTION

M5 · THE STACK, AND WHAT IT COSTS

The full tools register is `brand/site/tools.js`, gated by `brand/site/checktools.js`, which fails the build if any paid tool does not name **both** the free option it replaces and the concrete condition that forces the upgrade. "When we grow" is rejected by the checker; a number or a named event passes.

19 capabilities. Only three have no free path in existence:

Capability	Why there is no free option	What it costs
WhatsApp Business messaging	Meta requires an approved provider; there is no free tier to outgrow	~₹1,500–3,000/month plus per-conversation charges
SMS	Carrier access is the cost	~₹0.15–0.25 per message
Generated imagery	Free software, but it needs a graphics card	A GPU, or a hosted API per image

Everything else runs on free tiers or free software until a stated trigger fires: Postgres, self-hosted automation, a free hosting tier, local AI, the browser's own speech synthesis, ffmpeg and Chromium for media, UPI for payments, CSV for every integration, and a progressive web app instead of a store listing. **A business can run the whole system on zero rupees of software licence** and pay only for the three lines above, and only when it reaches them.

The Provider Router in Module 01 puts a spend ceiling in front of every paid call and refuses past it rather than warning — so the AI line in particular cannot quietly become the largest one.

M6 · THE EIGHT PHASES

The gate is absolute: **Phase N+1 does not start until Phase N's tests pass.** A phase is done when its stated result is reproduced, not when its code is written.

Phase	What gets built	Done when
0 · Setup	Environments, CI, monitoring, the schema loaded	A commit deploys and a user logs in
1 · Foundation & tenancy	Tenants, companies, roles, RLS, masters, the SKU/item model	Two tenants exist and neither can read a single row of the other, proved by a test that tries
2 · The industry pack engine ✅	Vocabulary, stages, fields, documents and starting data as rows	**Done.** A seventh trade is added **without writing code**, during the test run, from a JSON string — `core/tests/packs.test.js` §4
3 · Core operations	Inventory, procurement, production, quality, warehouse	Three trades run the same operations screens on their own vocabulary
4 · Commerce & channels	Sales all channels, OMS, returns, logistics, settlement	A week of orders across channels, settled and reconciled
5 · Finance	Double-entry, tax, banking, period locks	A month closes: trial balance ties and returns generate from vouchers
6 · The AI layer	Assistant, chatbot, agents, guardrails, content	An assistant answer matches the books; an agent asked to move money stops
7 · Onboarding & self-serve	Sign-up, pack selection, import, go-live	A business onboards itself in a day without a call

Phase 2 is the one that matters, and it is the one that is finished. The gate was written before the engine was — *a new trade must be addable without a developer* — and it is now run on every test pass against a trade nobody designed for. Phase 3 onwards is built on top of a claim that has been checked rather than one that was argued past.

M7 · ONBOARDING A BUSINESS IN A DAY

For a business with no system to migrate from — which is most of the target market — the sixty-day parallel run in the Vastrangam plan is the wrong shape entirely. The sequence is:

1. **Sign up, pick a trade.** The industry pack loads vocabulary, stages, documents and a chart of accounts. Nothing is blank.
2. **Name the company.** One row. A group adds more rows now or later.
3. **Import what exists** — customers, suppliers, items, opening stock — from a spreadsheet, with a validation report **before** anything commits. Errors are shown as rows to fix, never silently skipped.

4. **Opening balances**, or none at all if the business is starting fresh.
5. **Invite people, set roles.** Permissions are per company per role from the first minute.
6. **Do one real transaction end to end** — one sale, invoiced, stock moved, posted — and click the dashboard figure down to the voucher. That is the go-live test, and it takes a minute.

For a business migrating from an existing system, the discipline in `PLAN_OF_ACTION.md` Part IV E5 applies unchanged: export, clean, load, opening balances, parallel run, and a decommission criterion agreed in advance rather than argued at the end.

M8 · SECURITY AND COMPLIANCE

Row-level security in the database and permission checks in API middleware — one layer is one mistake away from a cross-tenant read. Personal data is exportable and erasable on request, with consent and retention tracked as the two separate clocks they are. Card data never reaches this system; the gateway's own secured field takes it, so there is no scope to protect. The audit trail has no off switch, and R16.22 keeps individual pay and personal attributes out of any document that leaves the building.

M9 · PERFORMANCE

Operation	p95
Page load, cached / uncached	< 1 s / < 3 s
Live stock or availability query	< 500 ms
Document PDF	< 2 s
Channel order pull, per channel	< 60 s
Settlement import, 1,000 lines	< 30 s
Daily profit-and-loss refresh	< 10 s

The availability query is the one that decides whether people trust the system: it runs on every screen showing a quantity or a free slot, and one that takes two seconds is one people stop asking.

M10 · RISK REGISTER

Risk	Likelihood	Impact	Mitigation
Cross-tenant data leak	Low	**Critical**	RLS with USING and WITH CHECK, middleware checks, a test that actively attempts a cross-tenant read
The industry pack engine slips, and trades get hard-coded instead	**High**	**Critical**	Phase 2 gate: a third trade added with no code. Hold the gate
A vertical demands a genuine structural exception	Medium	High	Add it as a module or an app for everyone, never as a branch for one customer
Free tier terms change or a tier disappears	Medium	Medium	Every capability has a self-host route recorded; the register is dated
WhatsApp provider outage	Low	High	Adapter swap; SMS and in-app as fallback
Onboarding needs hand-holding, so it does not scale	High	Medium	The day-one sequence above is a tested path, not a document
AI spend runs away	Medium	Medium	The spend ceiling refuses rather than warns

M11 · SUCCESS METRICS

Measure	Target
A new trade supported without writing code	Yes, from Phase 2 onward — binary, not a percentage
Time for a business to onboard itself	Under a day, unattended
Cost to run for a business at pilot scale	₹0 in software licence
Rules enforced by a test	the enforced-of-total figure at the head of this document; up every build
Cross-tenant incidents	Zero, and this is the only metric with no acceptable non-zero value

M12 · RUNBOOKS

Daily — error and uptime check, failed integrations queue, onboarding funnel. **Weekly** — usage per tenant, slow queries, support themes that suggest a missing rule. **Monthly** — free-tier headroom against triggers, spend against ceilings, backup restore test. **Quarterly** — re-read the tools register against current published tiers, review which SPECIFIED rules became enforceable, and check whether any vertical has quietly grown an exception.

M13 · WHAT A CUSTOMER PAYS FOR

The free-first register is not only how Medhava is built — it is the shape of what it can offer. Because the system runs on free tiers and free software at pilot scale, a business can be given a genuinely working system before it pays anything, and the paid lines are the ones it would have had to pay anyway: messaging, marketplace access, payment processing, courier pickup.

The commercial decision — subscription, per-company, per-user, or usage — is not made in this document, because it is a business decision rather than an engineering one. What this document fixes is the constraint it must respect: **nothing in the architecture may make a plan cap technically necessary.** Companies, channels, users and trades are rows. If a plan limits them, that is a commercial choice stated in the pricing, and the software says so rather than pretending to a technical limit it does not have.

PART III — THE PROOF

The test of whether this is one system or twenty-two programs sharing a login is a single transaction followed end to end, and it is re-run at every module boundary. That walkthrough is in `PLAN_OF_ACTION.md` Part V and is identical here.

The test of whether it is *industry-neutral* is different, and it is this: **take the same transaction and run it in a trade nobody had in mind when the code was written.** That test used to be run by hand, by writing an overlay. It is now run by the machine: `core/tests/packs.test.js` invents a commercial laundry every time it executes, loads it from a JSON string, and requires the system to answer in that trade's words — while still refusing it the audit trail. The last assertion in that file is that `core/packs.js` contains no trade word at all, so the engine cannot quietly learn one trade and call it neutrality.

Every check passes, with the shipped packs and a seventh added at run time. That is the whole industry-neutrality claim, reduced to something that either passes or does not.

Counts in this document are read from `brand/site/modules.js`, `brand/site/rules.js`, `brand/site/shots.js`, `brand/site/tools.js` and `core/schema.postgres.sql` when it is generated. Product screens carry illustrative figures and are labelled with the trade they are drawn from; they are not case studies, and no business is named as a customer that is not one.
